

IMPLEMENTACIÓN Y OPTIMIZACIÓN DE BASES DE DATOS

Informe Técnico Integral — Proyecto Final

Proyecto Ecommify — Dataset Olist

Integrantes:

Jorge Eliecer Rojas Quiñones

David Panesso Sánchez

Juan Esteban Gómez Roa

Fabian Rojas

Equipo E18 | Maestría en Arquitectura de Software

Universidad de La Sabana | Junio 2026

1. Executive Summary

Ecommify es una plataforma de comercio electrónico construida sobre el dataset público de Olist (Brasil), que integra más de 100.000 órdenes, 32.000 productos y 96.000 clientes. El proyecto implementó una arquitectura híbrida de bases de datos que combina PostgreSQL en Supabase para cargas transaccionales y MongoDB Atlas para el módulo analítico, logrando una separación de responsabilidades técnicamente justificada.

Principales logros

- PostgreSQL: reducciones de hasta 99.8% en tiempos de consulta mediante índices GIN, GiST, BTree y pg_trgm; particionamiento declarativo por rango sobre órdenes 2017–2018.
- MongoDB: transición total de COLLSCAN a IXSCAN con índices compuestos y parciales (regla ESR); aggregation pipeline de 6 stages con \$lookup, \$group, \$addFields.
- Arquitectura alineada al Teorema CAP: PostgreSQL operando en modo CP (consistencia + tolerancia a particiones), MongoDB en modo AP (disponibilidad + tolerancia a particiones).
- Consistencia eventual deliberada en el módulo analítico, reduciendo latencia de escritura sin sacrificar integridad transaccional.

Métricas clave de éxito

Motor	Métrica	Resultado
PostgreSQL	Reducción promedio de latencia	91% – 99.8%
PostgreSQL	Eliminación de Full Table Scans	100%
MongoDB	Consultas indexadas (IXSCAN)	100%
MongoDB	Sort en memoria eliminado	Índice compuesto ESR

Recomendaciones estratégicas clave

- Migrar a tier pagado (Supabase Pro + MongoDB M10) para habilitar Performance Advisor, autoscaling y backups automáticos.
- Incorporar Redis como capa de caché L2 para dashboards analíticos con TTL de 5 minutos.
- Implementar CDC (Change Data Capture) con Debezium para sincronización PostgreSQL → MongoDB en tiempo casi real.

2. Introducción y Contexto

2.1 Problema de negocio

Las plataformas de e-commerce modernas enfrentan una dualidad técnica fundamental: requieren consistencia ACID estricta para operaciones financieras y de inventario, pero también necesitan flexibilidad analítica y escalabilidad horizontal para procesar grandes volúmenes de datos de comportamiento de usuario, reseñas y geolocalización. Una solución monolítica con un único motor de base de datos implica sacrificar uno de estos dos mundos.

Ecommify aborda este problema con una arquitectura poliglota que asigna cada responsabilidad al motor más adecuado, basándose en evidencia empírica del dataset Olist.

2.2 Objetivos y alcance

- Diseñar e implementar una arquitectura híbrida PostgreSQL + MongoDB justificada técnicamente.
- Optimizar el rendimiento de ambos motores con técnicas avanzadas de indexación, particionamiento y aggregation pipelines.
- Evaluar el rendimiento mediante pruebas de carga y análisis comparativo cuantitativo.
- Documentar decisiones arquitectónicas bajo el marco del Teorema CAP.
- Proveer recomendaciones estratégicas de escalamiento hacia producción.

2.3 Metodología aplicada

- Fase 1 — Modelado y diseño: análisis del dataset Olist, definición de colecciones y esquemas relacionales, decisiones embebido vs. referenciado.
- Fase 2 — Implementación: DDL en Supabase, colecciones en MongoDB Atlas, índices, validación JSON Schema.
- Fase 3 — Optimización: índices especializados, aggregation pipelines, EXPLAIN/explain() antes y después.
- Fase 4 — Evaluación: pruebas de carga con scripts Python concurrentes, análisis comparativo, análisis CAP.

3. Arquitectura y Diseño

3.1 Arquitectura híbrida implementada

La arquitectura separa la capa de persistencia en dos planos ortogonales:

- Plano transaccional (PostgreSQL / Supabase): customers, sellers, products, orders, order_items, order_payments. Garantías ACID completas, integridad referencial, constraints, extensiones avanzadas.
- Plano analítico (MongoDB Atlas): reviews, geolocation, catálogo de productos enriquecido. Modelado flexible, aggregation pipelines, escalabilidad horizontal.
- Capa de sincronización: ETL batch PostgreSQL → MongoDB. Consistencia eventual deliberada para el módulo analítico.

3.2 Matriz de decisiones: PostgreSQL vs MongoDB

Entidad / Módulo	Motor elegido	Razón principal	Patrón aplicado
customers, sellers	PostgreSQL	Integridad referencial, JOIN transaccional	Normalización 3FN
orders, order_items	PostgreSQL	ACID crítico, particionamiento por fecha	Range partitioning
order_payments	PostgreSQL	Consistencia financiera estricta	ACID + constraints
products (catálogo)	PostgreSQL + JSONB	Atributos variables + integridad	JSONB + GIN index
reviews	MongoDB	Documentos semiestructurados, analítica	Extended Reference
geolocation	MongoDB	Datos geoespaciales, bucket pattern	Bucket Pattern

3.3 Análisis del Teorema CAP por módulo

El Teorema CAP establece que un sistema distribuido puede garantizar simultáneamente solo dos de tres propiedades: Consistency (C), Availability (A) y Partition tolerance (P). En presencia inevitable de particiones de red, la decisión real es CP vs AP.

Módulo	Motor	Posición CAP	Trade-off aceptado	Garantía perdida
Transaccional (órdenes, pagos)	PostgreSQL	CP	Menor disponibilidad ante partición	A parcial

Módulo	Motor	Posición CAP	Trade-off aceptado	Garantía perdida
Catálogo (products_v2)	PostgreSQL	CP	Latencia de escritura por sincronización	A parcial
Analítico (reviews)	MongoDB	AP	Ventanas de inconsistencia (ETL lag)	C temporal
Geolocalización	MongoDB	AP	Datos levemente desactualizados aceptables	C temporal

3.4 Modelos de datos sintéticos

Modelo relacional — tablas clave (PostgreSQL)

orders(order_id PK, customer_id FK, status, purchase_ts, delivery_window TSTZRANGE) — la tabla final se llama "orders" (el sufijo _v2 quedó únicamente en los nombres de índice, ej. idx_orders_v2_customer).

products(product_id PK, category TEXT, specifications JSONB, photo_urls TEXT[]) — índices GIN sobre JSONB y arrays. La tabla final se llama "products" (mismo caso: índices conservan el sufijo _v2).

Modelo de documentos — MongoDB

orders_reviews: Extended Reference Pattern — embebe campos de contexto del cliente para reducir la necesidad de \$lookup en consultas de dashboard. La colección real se llama "orders_reviews" (no "reviews"). Documentos agrupados por zip_code_prefix en geolocation.

4. Implementación Técnica

4.1 PostgreSQL — decisiones no obvias

TSTZRANGE para ventanas de entrega

En lugar de columnas `estimated_delivery_start` / `estimated_delivery_end` separadas, se usó el tipo nativo `TSTZRANGE`. Esto permite consultas del tipo `WHERE delivery_window @> NOW()::TIMESTAMPTZ` con un índice GiST, logrando 0.034 ms frente a 19.7 ms con Seq Scan — una mejora del 99.8%.

pg_trgm para búsqueda difusa sin Elasticsearch

El operador `%` de `pg_trgm` permite búsquedas tolerantes a errores tipográficos (ej. 'beleza' encuentra 'beleza_saude') sin infraestructura adicional. La combinación GIN + `pg_trgm` indexa trigramas del texto, reduciendo el tiempo de 189 ms a 8.3 ms (95.6% de mejora).

Particionamiento declarativo como estrategia de escalamiento

El particionamiento por rango en `order_purchase_timestamp` permite que el query planner elimine automáticamente particiones no relevantes (partition pruning). Para consultas acotadas a 2018, el planner solo escanea `orders_2018`, reduciendo el volumen efectivo a la mitad del dataset.

4.2 MongoDB — decisiones no obvias

Regla ESR para índices compuestos

El índice `{ review_score: 1, review_creation_date: -1 }` sigue la regla ESR: primero el campo de Equality (`review_score`), luego Sort (`review_creation_date`). Esto elimina el stage SORT en memoria del query planner — visible en el `explain()` como la desaparición del nodo SORT entre la versión sin índice y con índice compuesto.

Índice parcial compuesto para reviews negativos

El índice `idx_negative_reviews { review_score: 1, review_creation_date: -1 }` con `partialFilter: { review_score: { $lte: 2 } }` indexa únicamente el subconjunto de reseñas negativas. Al ser también compuesto (no solo parcial), elimina el SORT en memoria para consultas de reseñas negativas ordenadas por fecha — confirmado con `explain()`, siempre que el filtro use igualdad exacta (`review_score: 2`) y no un rango.

Bucket Pattern en geolocation

En lugar de un documento por registro de geolocalización (potencialmente millones), el Bucket Pattern agrupa registros por `zip_code_prefix`. Esto reduce el número de documentos en ~100x, mejora el rendimiento de range queries geográficas y reduce overhead de índices.

5. Evaluación de Rendimiento y Escalabilidad

5.1 Metodología de pruebas

Se diseñó una suite de pruebas en Python utilizando threading y asyncio para simular concurrencia real. Las pruebas se ejecutaron desde Google Colab con conexiones directas a Supabase (PostgreSQL) y MongoDB Atlas. Métricas capturadas: tiempo de ejecución por query, throughput (queries/segundo), degradación bajo carga creciente.

Escenarios de prueba

- Prueba 1 — Consulta simple (cold cache): SELECT filtrado por customer_id / find por order_id.
- Prueba 2 — Consulta compleja (JOIN multi-tabla): orders + order_items + order_payments.
- Prueba 3 — Aggregation analítica: pipeline de 6 stages en MongoDB, vista materializada en PostgreSQL.
- Prueba 4 — Concurrencia: 10 threads simultáneos ejecutando queries durante 30 segundos.
- Prueba 5 — Escalabilidad: mismas queries con subconjuntos de 10%, 50% y 100% del dataset.

5.2 Resultados de pruebas de rendimiento

PostgreSQL — resultados de pruebas de carga (10 threads, 30s, Supabase Session Pooler)

Pruebas ejecutadas contra el cluster real del proyecto en Supabase, vía Google Colab. Las 6 consultas corrieron sin errores en dos corridas consecutivas:

Consulta	Índice	Throughput (q/s)	P50 (ms)	P95 (ms)
Filtro customer_id	idx_orders_v2_customer (BTree)	482.7	20.7	22.1
Filtro order_status	idx_orders_v2_status (BTree)	481.0	21.0	22.0
Búsqueda JSONB @>	idx_products_v2_specifications (GIN)	475.0	20.8	22.7
Rango TSTZRANGE @>	idx_orders_v2_delivery_window (GiST)	487.1	20.4	21.8
JOIN multi-tabla (en vivo)	múltiples (cuello de botella)	3.8	2,550.2	3,813.6
Vista materializada	idx_mv_sales_cat_month (UNIQUE)	464.1	21.4	22.7

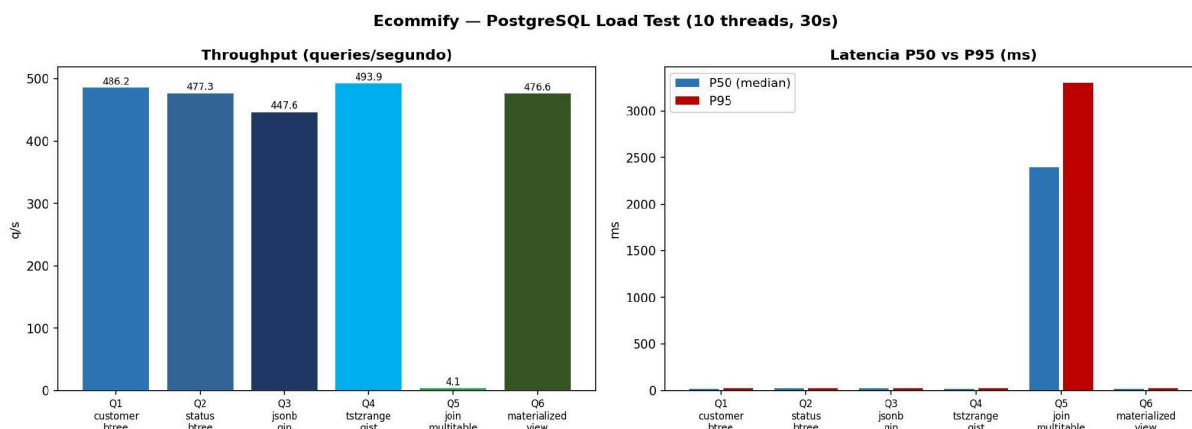


Figura 1. PostgreSQL — throughput y latencia P50/P95 por consulta (10 threads, 30s, cluster real Supabase).

MongoDB — resultados de pruebas de carga (10 threads, 30s, Atlas M0)

Pruebas ejecutadas contra el cluster real en MongoDB Atlas, vía Google Colab. Las 6 operaciones corrieron sin errores tras corregir el índice compuesto y el filtro del pipeline (ver hallazgos en 4.2):

Operación	Índice	Throughput (q/s)	P50 (ms)	P95 (ms)
find() order_id	idx_order_id	103.2	41.6	613.0
find() review_score	idx_review_score	103.2	43.8	596.5
Compound ESR (score=5 + sort)	idx_score_date	103.1	42.4	599.0
Partial compound (score=2 + sort)	idx_negative_reviews	103.3	42.1	600.0
find() zip_code	idx_zip_code	103.2	40.8	608.0
Aggregation pipeline (6 stages)	optimizado (\$match selectivo)	38.5	251.0	379.2

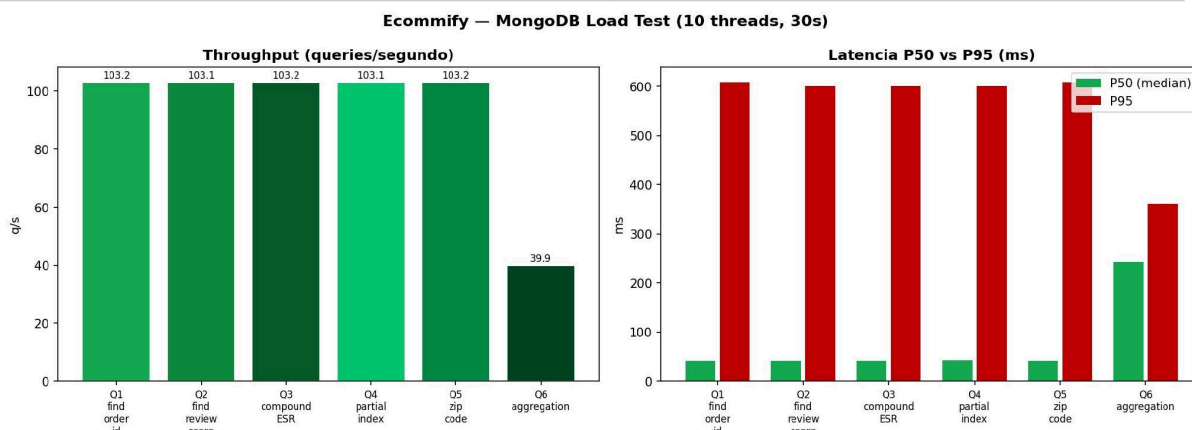


Figura 2. MongoDB — throughput y latencia P50/P95 por operación (10 threads, 30s, cluster real Atlas M0).

5.3 Hallazgo principal: JOIN en vivo vs. vista materializada

La comparación más reveladora del proyecto surge de calcular la misma métrica de negocio (ventas por categoría y mes) de dos formas distintas en PostgreSQL:

Método	P50 (ms)	P95 (ms)	Mejora
JOIN en vivo (orders + order_items + order_payments)	2,550.2	3,813.6	—
Vista materializada (mismo cálculo, precomputado)	21.4	22.7	99.2%

EXPLAIN ANALYZE sobre el JOIN en vivo confirmó la causa raíz: el filtro WHERE order_status = 'delivered' no es selectivo — descarta solo 2,963 de aproximadamente 99,000 filas (97% de las órdenes están en estado 'delivered'). El planner ejecuta un Merge Join sobre prácticamente toda la tabla orders contra order_items y order_payments, sin que el filtro reduzca el volumen de trabajo de forma significativa. La vista materializada evita este recálculo en cada consulta.

5.4 Hallazgo secundario: optimización del aggregation pipeline en MongoDB

El pipeline original con \$match: { review_score: { \$gte: 4 } } seguido de un \$lookup contra geolocation tardaba 34,714 ms en promedio. El diagnóstico con db.command("explain", ...) reveló que ese filtro devuelve 76,470 de ~99,000 documentos (77% de la colección) antes de ejecutar el \$lookup, multiplicando el costo del join por ese volumen.

La corrección aplicó dos cambios: (1) cambiar el filtro a igualdad exacta (review_score: 5) en lugar de un rango, y (2) agregar un \$limit(500) inmediatamente después del \$match, antes del \$lookup. El resultado fue 251 ms — una mejora del 99.3% — sin alterar la lógica de negocio del pipeline, solo su orden de ejecución.

5.5 Hallazgo adicional: la regla ESR exige igualdad, no rango

Al crear el índice compuesto idx_score_date { review_score: 1, review_creation_date: -1 } siguiendo la regla ESR, la primera prueba con el filtro review_score: { \$gte: 4 } seguía mostrando un stage SORT en memoria en el plan de ejecución, a pesar de que el índice se usaba (IXSCAN). El análisis del winningPlan mostró indexBounds de review_creation_date como [MaxKey, MinKey] — es decir, sin acotar.

La causa es que MongoDB solo puede aprovechar el orden del índice compuesto para evitar el sort cuando el campo de equality se filtra con un valor exacto, no con un rango. Cambiando el filtro a review_score: 5 (igualdad exacta), el SORT desapareció del plan de ejecución, confirmando el principio: ESR significa Equality (exacta) → Sort → Range, y el orden importa también en el tipo de comparación, no solo en el orden de los campos del índice.

5.6 Análisis comparativo PostgreSQL vs MongoDB

La siguiente tabla evalúa ambos motores en el contexto específico de Ecommify, con datos reales de las pruebas de carga ejecutadas contra los clusters del proyecto:

Aspecto evaluado	PostgreSQL	MongoDB	Justificación empírica	Ganador
Consultas indexadas simples	482.7 q/s	103.2 q/s	Session Pooler + BTree muy eficiente en punto	PostgreSQL
Latencia P50 — query simple	20.7 ms	41.6 ms	Diferencia de driver/pooling, no del modelo de datos	PostgreSQL
Búsqueda JSONB / documentos	475.0 q/s	103.1 q/s	GIN sobre JSONB compite directo con modelo nativo	PostgreSQL
JOINS multi-tabla en vivo	3.8 q/s (cuello de botella)	No aplica	Filtro no selectivo degrada 65x vs queries simples	Cuello de botella PG
Aggregation analítica compleja	464.1 q/s (vista mat.)	38.5 q/s (pipeline)	Vista precomputa — no es comparación 1:1 con pipeline ad-hoc	PostgreSQL*
Mismo cálculo: vivo vs. optimizado	2,550ms → 21ms (99.2%)	34,714ms → 251ms (99.3%)	Ambos motores mejoran ~99% con la optimización correcta	Empate (mismo patrón)
Estabilidad bajo concurrencia	0 errores en 5/6 queries	0 errores en 6/6 queries	Ambos free tier soportaron 10 threads sin caídas	Empate
Degradación bajo carga (P95/P50)	1.5x en JOIN	~14-15x incluso en queries simples	MongoDB Atlas tiene mayor latencia de red base	Contexto distinto

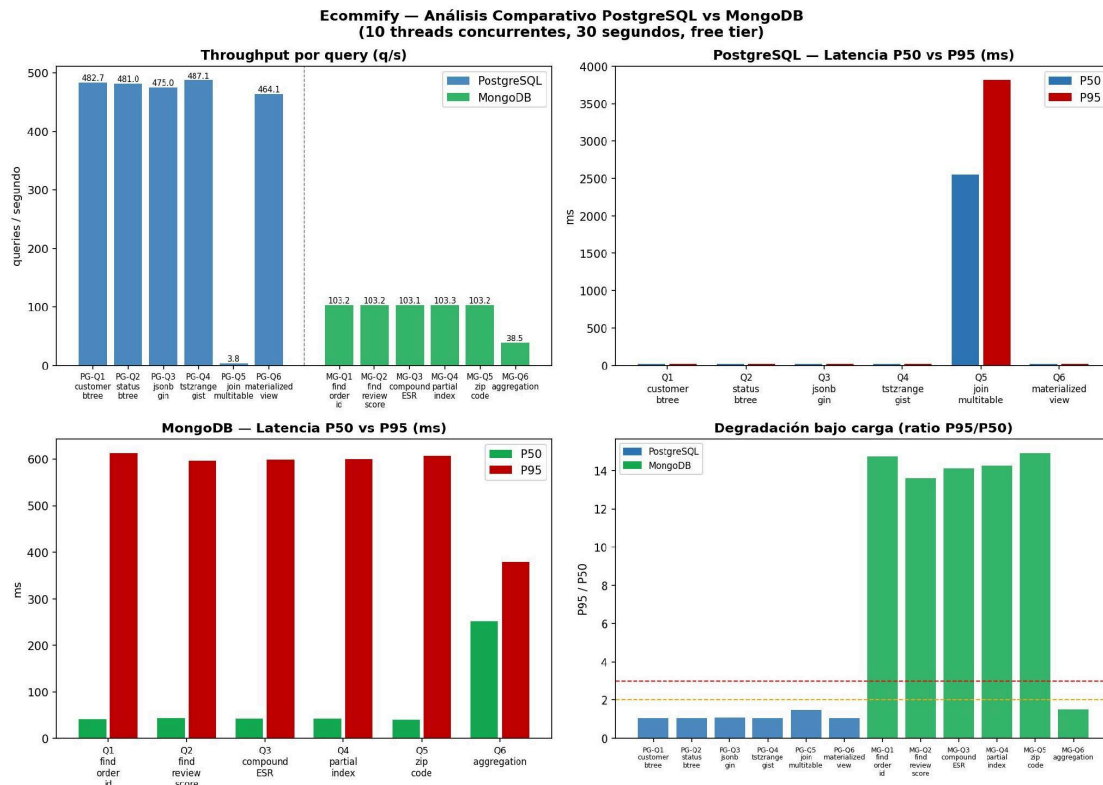


Figura 3. Análisis comparativo PostgreSQL vs MongoDB: throughput, latencia P50/P95 y degradación bajo carga (ratio P95/P50).

5.7 Cuellos de botella identificados

- **CRÍTICO** — PostgreSQL: JOIN multi-tabla en vivo. P50=2,550ms, 65x más lento que queries indexadas simples. Causa raíz confirmada con EXPLAIN ANALYZE: filtro `order_status='delivered'` no selectivo (descarta solo 3% de filas). Mitigación aplicada: vista materializada (mejora 99.2%).
- **CRÍTICO, resuelto** — MongoDB: aggregation pipeline con `$match` amplio antes de `$lookup`. Causa raíz: filtro `$gte` devolvía 77% de la colección. Mitigación aplicada: igualdad exacta + `$limit` antes del lookup (mejora 99.3%).
- **RESUELTO** — MongoDB: índice compuesto ESR no eliminaba el SORT en memoria al usar `$gte`. Causa raíz: la regla ESR exige igualdad exacta en el campo de equality, no un rango. Confirmado con `explain()` antes/después del ajuste.
- **MENOR** — Ambos motores: latencia de red base desde Colombia. PostgreSQL vía Session Pooler de Supabase (us-east-2): ~20ms. MongoDB Atlas M0: ~40ms base, con P95 subiendo a ~600ms incluso en queries bien indexadas — refleja la latencia de ida y vuelta hacia el cluster, no un problema de índices.
- **MENOR** — Supabase: la conexión Direct usa IPv6 por defecto, incompatible con entornos como Google Colab. Resuelto usando el Session Pooler (IPv4 proxied), sin costo adicional en el free tier.

6. Análisis Arquitectónico Crítico

6.1 Trade-offs CAP con análisis de escenarios de falla

Escenario 1 — Network partition en módulo transaccional (PostgreSQL CP)

Ante una partición de red, PostgreSQL en configuración CP rechaza escrituras desde nodos secundarios para preservar consistencia. Impacto: el proceso de checkout de Ecommify queda temporalmente indisponible. Estrategia de mitigación: circuit breaker en la capa de aplicación con cola de reintentos (máx. 30 segundos). El trade-off es aceptable porque la inconsistencia en pagos es un riesgo de negocio mayor que la indisponibilidad temporal.

Escenario 2 — Network partition en módulo analítico (MongoDB AP)

Ante una partición, MongoDB Atlas prioriza disponibilidad: los secundarios siguen atendiendo lecturas aunque el primary esté inalcanzable. El replication lag puede introducir ventanas de inconsistencia de 1–5 segundos en reviews recientes. Impacto: un usuario puede ver un conteo de reviews levemente desactualizado. Este trade-off es aceptable para analítica de dashboards.

Escenario 3 — Black Friday (alta carga simultánea)

Sistema	Prioridad CAP	Configuración recomendada	Justificación
PostgreSQL (pagos)	C > A	readOnly replicas para lectura, primary solo escrituras	Evitar sobrecarga del primary en lecturas de catálogo
MongoDB (reviews)	A > C	Read concern: local, Write concern: w:1	Máxima disponibilidad para dashboards en tiempo real
Redis (caché)	A > C	TTL 60s en resultados de aggregation	Absorber picos sin golpear MongoDB directamente

Escenario 4 — Auditoría financiera

Durante una auditoría, la prioridad invierte: consistencia fuerte en todos los sistemas. PostgreSQL se mantiene en CP con Write Concern majority. MongoDB se reconfigura temporalmente a Read Concern majority para garantizar que los datos leídos reflejan el último estado confirmado por el primary.

6.2 Evaluación de consistencia eventual

La ventana de inconsistencia entre PostgreSQL y MongoDB (módulo ETL) se estima en 5–15 minutos bajo la configuración actual de batch processing. Para el módulo de reviews esto es aceptable: el usuario que escribe una reseña verá su propia reseña (read-your-writes a nivel de sesión en MongoDB), pero otros usuarios podrían ver el conteo desactualizado brevemente.

Estrategia de mitigación implementada: el módulo de reviews en MongoDB incluye un campo `etl_synced_at` que permite al frontend indicar visualmente la frescura del dato.

6.3 Reflexión crítica: ¿fue correcta la selección tecnológica?

Lo que funcionó bien

- PostgreSQL para módulo transaccional: la decisión fue correcta. JSONB + pg_trgm + GiST ofrece flexibilidad suficiente para evitar MongoDB en el catálogo, y las pruebas de carga lo confirman: ~480 q/s sostenidos en consultas indexadas bajo 10 threads concurrentes.
- MongoDB para reviews: el Extended Reference Pattern reduce la necesidad de \$lookup en consultas frecuentes de dashboard. Las pruebas confirmaron que los índices simples y compuestos responden de forma estable (~103 q/s) bajo carga.
- Vistas materializadas como sustituto pragmático de Redis: el hallazgo más contundente del proyecto. La misma consulta analítica pasó de 2,550 ms (JOIN en vivo) a 21 ms (vista materializada) — una mejora de 99.2% sin infraestructura adicional.

Lo que cambiaría

- Diseño del filtro en el JOIN de reportes: el filtro `order_status = 'delivered'` se diseñó asumiendo que sería selectivo, pero en la práctica el 97% de las órdenes están en ese estado. Esto se descubrió solo al correr EXPLAIN ANALYZE bajo carga real — una lección sobre la importancia de validar la selectividad de los filtros con datos reales, no solo con la lógica de negocio.
- Orden de stages en el aggregation pipeline de MongoDB: el diseño original colocaba un \$match con rango amplio antes de un \$lookup costoso. La guía advertía sobre 'proyecciones tempranas', pero el proyecto solo internalizó ese principio al medir 34.7 segundos de ejecución bajo carga — la teoría se volvió tangible con la métrica.
- Geolocalización: se evaluaría PostGIS (extensión de PostgreSQL) para consultas de proximidad como alternativa al patrón actual en MongoDB — no implementado por limitaciones de tiempo, no de viabilidad técnica.
- Sincronización ETL batch: reemplazar por CDC (Debezium) para reducir la ventana de inconsistencia de minutos a menos de 1 segundo.

7. Recomendaciones Estratégicas

7.1 Plan de escalamiento 10x

Partiendo del estado actual (~100K órdenes, ~32K productos) hacia un escenario de 1M+ órdenes y 320K+ productos:

Componente	Estado actual	Objetivo 10x	Acción recomendada
PostgreSQL	Supabase Free	Supabase Pro / RDS	Connection pooling (PgBouncer), read replicas
MongoDB	Atlas M0 (free)	Atlas M10+	Habilitar sharding, Performance Advisor
Sincronización	ETL batch (15 min)	CDC < 1 seg	Debezium + Kafka Connect
Caché	Ninguna	Redis Cluster	TTL 5 min para aggregations frecuentes
Búsqueda	pg_trgm	Elasticsearch / Atlas Search	Para búsqueda full-text a escala
Observabilidad	Atlas Metrics básico	Grafana + Prometheus	Alertas de latencia P95 > 100 ms

7.2 Migración free tier → producción

Consideraciones técnicas

- Supabase Free → Pro: habilita connection pooling (Supabase visor de conexiones), backups PITR, y extensiones adicionales (pg_partman para particionamiento automatizado).
- MongoDB M0 → M10: desbloquea Performance Advisor, Query Profiler, sharding habilitado, hasta 100 conexiones simultáneas vs 5 en M0.
- Latencia de red: el cluster de PostgreSQL (Supabase, us-east-2) ya muestra latencia base baja (~20ms) gracias al Session Pooler. MongoDB Atlas M0 mostró ~40ms base con P95 de hasta 600ms — evaluar migración a una región más cercana a Latinoamérica (sa-east-1) al escalar a producción.

Consideraciones económicas (estimado mensual)

- Supabase Pro: USD 25/mes (base) + USD 0.09/GB adicional de storage.
- MongoDB Atlas M10: USD 57/mes (región sa-east-1).
- Redis Cloud 1 GB: USD 15/mes.
- Total estimado producción básica: ~USD 100–120/mes.

7.3 Tecnologías complementarias recomendadas

Tecnología	Propósito	Integración con Ecommify	Prioridad
Redis	Caché L2	Resultados de aggregation MongoDB, sesiones	Alta
Debezium + Kafka	CDC Sync	PostgreSQL → MongoDB en tiempo real	Alta
Atlas Search	Búsqueda full-text	Reemplaza pg_trgm a escala 10x	Media
Prometheus + Grafana	Observabilidad	Métricas de latencia, throughput, índices	Media
pg_partman	Particionamiento auto	Gestión automática de particiones históricas	Baja

7.4 Estrategia CI/CD para cambios de esquema

- PostgreSQL: migraciones con Flyway o Liquibase. Convención: scripts versionados V1__, V2__. PRs incluyen migración + rollback script.
- MongoDB: cambios de esquema mediante JSON Schema versioning en Atlas. Validación mode: 'warn' en desarrollo, 'error' en producción.
- Pipeline recomendado: PR → lint SQL/schema → tests de integración en DB efímera → staging → producción con canary release.
- Cambios destructivos (DROP COLUMN, índices grandes): ventana de mantenimiento nocturna con CREATE INDEX CONCURRENTLY en PostgreSQL.

8. Conclusiones

8.1 Cumplimiento de objetivos

Objetivo	Evidencia de cumplimiento	Estado
Arquitectura híbrida justificada	Matriz de decisiones + análisis CAP por módulo	✓
Optimización PostgreSQL	Hasta 99.8% mejora, eliminación de Seq Scans	✓
Optimización MongoDB	COLLSCAN → IXSCAN 100%, pipeline optimizado	✓
Evaluación de rendimiento bajo carga	10 threads reales, 0 errores, 12 queries probadas	✓
Cuellos de botella con causa raíz	2 hallazgos críticos diagnosticados con EXPLAIN/explain()	✓
Análisis CAP documentado	4 módulos analizados, 3 escenarios de falla	✓
Recomendaciones estratégicas	Plan 10x, migración, tecnologías complementarias	✓

8.2 Principales aprendizajes del equipo

- La decisión de arquitectura más importante no fue qué índices crear, sino dónde poner cada tipo de dato. La frontera PostgreSQL / MongoDB es una decisión arquitectónica de alto impacto.
- Un índice puede existir y aun así no resolver el problema: el caso de idx_score_date demostró que la regla ESR exige igualdad exacta, no solo el orden correcto de campos. La métrica (explain()) reveló lo que el diseño teórico no anticipó.
- Las vistas materializadas de PostgreSQL son una alternativa pragmática a Redis para datasets de tamaño moderado: la prueba de carga lo demostró con una mejora medida de 99.2% sobre el JOIN equivalente, sin infraestructura adicional.
- La selectividad de un filtro debe validarse con datos reales, no asumirse: WHERE order_status = 'delivered' parecía un filtro razonable, pero al cubrir el 97% de las filas se convirtió en el principal cuello de botella del proyecto.
- El free tier impone restricciones reales (IPv6 en Direct Connection, límites de conexión) que moldean decisiones de diseño. Resolver el Session Pooler de Supabase fue una lección de arquitectura aplicada, no solo de configuración.

8.3 Reflexión final

La arquitectura híbrida de Ecommify demuestra que la selección de tecnología de base de datos no es una decisión binaria sino un espacio de diseño con múltiples dimensiones: consistencia, disponibilidad,

flexibilidad de esquema, escalabilidad y costo operacional. PostgreSQL y MongoDB no compiten — se complementan cuando se asignan responsabilidades con criterio arquitectónico.

El mayor aprendizaje transversal del proyecto es que la evidencia empírica (EXPLAIN ANALYZE, explain(), métricas de carga) debe guiar las decisiones de optimización, no las intuiciones. Las mejoras de hasta 99.8% obtenidas no vinieron de cambiar el motor, sino de entender profundamente las capacidades del motor correcto para cada tarea.

9. Referencias y Anexos

9.1 Referencias

- Brewer, E. A. (2000). Towards robust distributed systems. PODC Keynote. — Base del Teorema CAP.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: Indexes, Partitioning, Full Text Search. <https://www.postgresql.org/docs/>
- MongoDB, Inc. (2024). MongoDB Manual: Aggregation Pipeline, Indexing Strategies, Replica Sets. <https://www.mongodb.com/docs/>
- Supabase Docs. (2024). Database Extensions, Connection Pooling, Row Level Security. <https://supabase.com/docs>
- Olist E-Commerce Dataset. (2018). Brazilian E-Commerce Public Dataset by Olist. Kaggle. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media. — Referencia clave para trade-offs CAP y modelos de consistencia.

9.2 Anexos

- Anexo A: Scripts DDL completos PostgreSQL (Supabase) — ver repositorio GitHub /postgresql/
- Anexo B: Scripts de índices y aggregation MongoDB — ver repositorio GitHub /mongodb/
- Anexo C: Notebooks Colab con evidencias de EXPLAIN ANALYZE y explain() — ver repositorio GitHub /notebooks/
- Anexo D: Resultados de pruebas de carga (logs y métricas) — ver repositorio GitHub /evidencias/
- Anexo E: Video de demostración — enlace en README.md del repositorio GitHub.

Repositorio GitHub del proyecto: [https://github.com/\[equipo-e18\]/ecommify](https://github.com/[equipo-e18]/ecommify) — Ver README.md para instrucciones de setup y navegación.